# Planejamento Lógico TaskTracker

**Bootcamp II** - Documento de apoio da Fase 2 (Entrega Intermediária)

*Nota: este planejamento foi redigido durante a Fase 2, para documentar a lógica implementada em* `src/main.py`.

## 1. Identificação e entendimento do problema

**Nome: Lucas Maubrigades Marotti  Curso: Ciência da computação  Unidade:** ASA NORTE

**Problema.** Quem não tem um sistema para organizar tarefas depende da memória ou de anotações soltas. Isso leva a esquecimentos, prazos perdidos e dificuldade de saber o que é mais urgente.

**Solução proposta.** O TaskTracker é um programa de linha de comando em Python que permite cadastrar tarefas com prioridade e data limite e visualizar todas elas de forma organizada, com validação dos dados digitados.

## 2. Decomposição de dados e regras

**Entradas** (digitadas pelo usuário):

| Campo | Tipo | Observação |
|---|---|---|
| Opção do menu | Texto | 1 (cadastrar), 2 (visualizar) ou 3 (sair) |
| Título | Texto | Obrigatório |
| Descrição | Texto | Livre; pode ficar vazia |
| Prioridade | Texto | Alta, Média (ou Media) ou Baixa |
| Data limite | Texto | Livre; sugestão DD/MM/AAAA |

**Saídas** (mostradas na tela):

- Menu principal com as três opções.
- Mensagens de erro quando título ou prioridade são inválidos.
- Mensagem de sucesso após cadastrar uma tarefa.
- Lista de tarefas com todos os campos, ou aviso de que não há tarefas cadastradas.

**Regras do sistema:**

R1. O título é obrigatório; entradas vazias ou só com espaços são rejeitadas e o programa pede novamente.
R2. A prioridade só aceita Alta, Média (ou Media) e Baixa, sem diferenciar maiúsculas; qualquer outro valor gera erro e nova tentativa.
R3. O status é definido automaticamente como Pendente no cadastro.
R4. Se não houver tarefas, a visualização informa isso de forma clara.
R5. O menu se repete até o usuário escolher Sair.

## 3. Receita lógica (algoritmo em português)

**Programa principal**

```
1. Criar uma lista vazia de tarefas.
```

```
2. Repetir:
   2.1 Mostrar o menu (1-Cadastrar, 2-Visualizar, 3-Sair).
   2.2 Ler a opção.
   2.3 Se for 1, executar o cadastro.
   2.4 Se for 2, executar a visualização.
   2.5 Se for 3, avisar que vai encerrar e parar de repetir.
   2.6 Caso contrário, avisar que a opção é inválida.
```

**Cadastro de tarefa**

```
1. Pedir o título.
2. Se o título estiver vazio (ou só com espaços), avisar e voltar ao passo 1.
3. Pedir a descrição.
4. Pedir a prioridade.
5. Se não for Alta, Média/Media ou Baixa, avisar e voltar ao passo 4.
6. Pedir a data limite.
7. Montar a tarefa com os cinco campos, com status "Pendente".
8. Guardar a tarefa na lista.
9. Mostrar mensagem de sucesso.
```

**Visualização das tarefas**

```
1. Acessar a lista de tarefas.
2. Se a lista estiver vazia, avisar que não há tarefas e terminar.
3. Para cada tarefa, mostrar número, título, descrição, prioridade,
   data limite e status.
```

## 4. Mapeamento arquitetural (Git/GitHub)

```
tasktracker-projeto/
|-- README.md                    # Apresentação e instruções de uso
|-- .gitignore                   # Ignora __pycache__/, .pyc e .venv/
|-- docs/
|   `-- planejamento_logico.pdf  # Este documento
`-- src/
    `-- main.py                  # Código-fonte do TaskTracker
```

**Histórico de commits planejado:** (1) commit inicial com README, .gitignore e este documento; (2) menu e loop principal em main.py; (3) validações de título e prioridade; (4) publicação no GitHub com push para origin main.

**O que espero aprender sobre controle de versão: Como criar uma boa documentação para meu código e como melhorar minha apresentaçao de projetos.**